

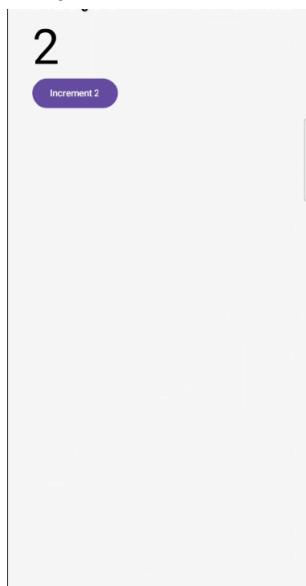
Nama : Ismail Tegar Rwatma Bhumi
NIM : 245150601111016

Link Github: <https://github.com/ismailtgr/TugasStateHoisting>

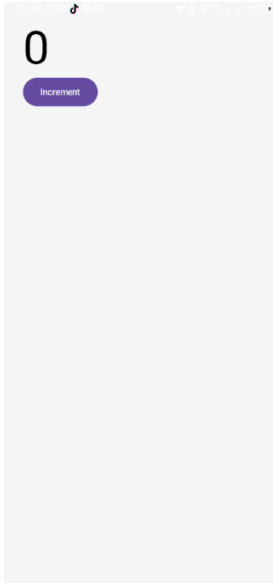
4.5.1 State Hoisting

Aktivitas Percobaan

1.
 - a. Angka yang ditampilkan pada Text akan bertambah 1 secara berurutan
 - b. Hal tersebut terjadi karena adanya perubahan nilai pada variabel state number saat tombol diklik (`number++`). Perubahan nilai state ini memicu proses **recomposition** (penggambaran ulang UI) pada Jetpack Compose untuk memperbarui tampilan Text dengan nilai angka terbaru.
 - c. Variabel state number.
 - d. Variabel state label.
2. As
 - a. Mengubah nilai kenaikan number yang berada di lambda onClick milik Button dari "`number++`" menjadi "`number += 2`". Kemudian mengubah label "Increment" menjadi "Increment 2" pada `mutableStateOf`.



b.



3.

4.

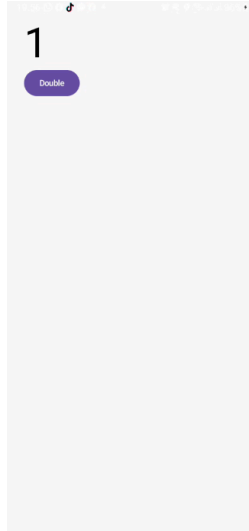
- a. **Ya**, program dapat berjalan dengan sangat baik dan tampilan serta perilakunya persis sama dengan tahap sebelumnya. Angka bertambah setiap kali tombol diklik. Perbedaannya hanya terletak pada arsitektur kodenya saja (manajemen state dipindahkan ke fungsi pemanggil/induk).
- b. Proses increment saat ini dilakukan di **dalam fungsi onCreate()** pada pemanggilan Composable CounterScreen, tepatnya di dalam lambda callback `onButtonClick = { number++ }`.
- c. State angka (number) dideklarasikan di **dalam fungsi onCreate()** menggunakan `var number by remember { mutableStateOf(0) }`, Label tombol ("Increment") di-pass sebagai **literal String langsung pada parameter label = "Increment"** di dalam fungsi `onCreate()`, Kedua nilai tersebut kemudian diteruskan ke bawah (*pass down*) sebagai argumen/parameter ke dalam fungsi Composable CounterScreen.

5.

- a.

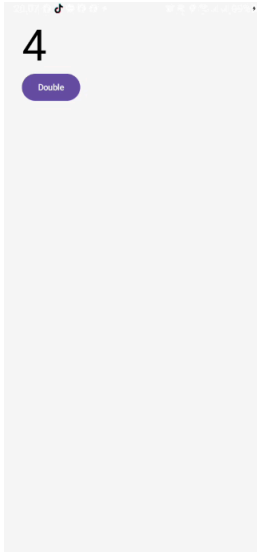
```
var number by remember { mutableStateOf(1) }
CounterScreen(
    modifier = Modifier.padding(32.dp),
    number = number,
    label = "Double",
    onButtonClick = { number *= 2 }
)
```

6.

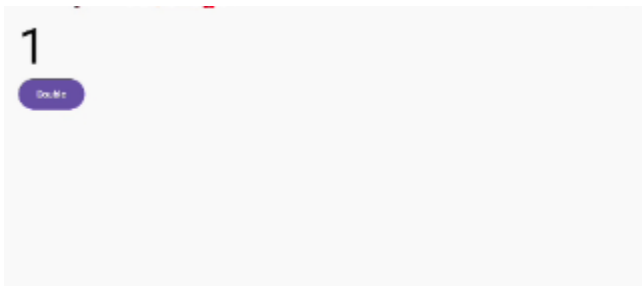


- a.
 - b. Seluruh perubahan dilakukan **hanya di dalam fungsi onCreate()** (pada level Composable induk / pemanggil). Perubahan meliputi inisialisasi nilai awal state number, teks literal pada parameter label, dan logika perkalian pada lambda onClick.
 - c. **Tidak ada perubahan sama sekali** pada fungsi Composable CounterScreen. Fungsi CounterScreen tetap menggunakan bentuk *signature* dan *body* yang sama persis seperti pada tahap nomor 7.
7. Hasil pada tahap nomor 9 membuktikan bahwa *state hoisting* memberikan manfaat besar berupa fleksibilitas dan keterpeliharaan kode (*maintainability*) yang tinggi, karena komponen UI CounterScreen kini bersifat *stateless* dan memiliki pemisahan tanggung jawab (*separation of concerns*) yang jelas dari logika bisnis. Perubahan aturan aplikasi—seperti mengganti logika penambahan angka menjadi perkalian maupun mengubah teks label—dapat dilakukan sepenuhnya pada komponen induk (onCreate) tanpa perlu menyentuh atau merisikokan kerusakan pada kode UI CounterScreen sama sekali, sehingga komponen antarmuka tersebut menjadi sangat relevan untuk digunakan kembali (*reusable*) di berbagai bagian aplikasi lainnya.

4.5.2 Perubahan Konfigurasi Aktivitas Percobaan

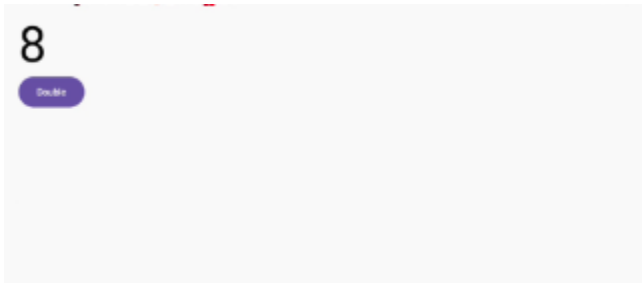


1.



2.

- a. Angka yang ditampilkan saat layar diputar ke posisi landscape akan **kembali ke angka awal, yaitu 1**.
- b. Angka yang ditampilkan adalah 1, hal tersebut dikarenakan saat posisi layar diputar terjadi *configuration change* seperti rotasi dari potrait ke landscape, android secara bawaan akan melakukan *destroy* dan *recreate*.



3.

- a. Angka yang ditampilkan akan **tetap mempertahankan nilai terakhir sebelum layar diputar**.
- b. Angka yang ditampilkan adalah 8, **Penyebabnya**, Saat terjadi perubahan konfigurasi (*configuration change*) akibat rotasi layar, Android memang menghancurkan dan membuat ulang *Activity*. Namun, karena kita menggunakan *rememberSaveable*, sistem menyimpan *state* variabel number ke dalam Bundle saat *Activity* dihancurkan, lalu memulihkan nilainya kembali saat *Activity* dibuat ulang.

- c. **Fungsi rememberSaveable:** Menyimpan *state* antarmuka pengguna agar tidak hilang ketika terjadi perubahan konfigurasi (*configuration change*) maupun saat proses aplikasi dihentikan sementara oleh sistem.
Perbedaannya `remember` hanya mempertahankan *state* selama proses **recomposition** (perubahan data biasa tanpa *recreate Activity*). Sedangkan `rememberSaveable` mempertahankan *state* baik saat **recomposition** maupun **recreation** (*Activity* dihancurkan dan dibuat ulang akibat rotasi layar), karena datanya disimpan secara otomatis ke dalam Bundle milik Android.
- 4. Form Pendaftaran atau Input Teks Panjang (misalnya input teks formulir alamat pengiriman pada aplikasi *e-commerce*, atau teks pencarian pada halaman filter). Manfaatnya adalah **mencegah Kehilangan Data pengguna (Preventing Data Loss)**, Jika pengguna sedang mengisi form yang panjang lalu tanpa sengaja memutar ponselnya, teks yang sudah diketik tidak akan terhapus secara mendadak. **Pengalaman Pengguna yang Lebih Baik (Better User Experience)**, Aplikasi terasa lebih stabil, responsif, dan seamless karena pengguna tidak perlu mengulang input data dari awal setiap kali terjadi perubahan orientasi layar.

4.5.3 LaunchedEffect

Aktivitas Percobaan

1. As
 - a. Layar menampilkan angka **1**, tombol **Double**, dan sebuah **Snackbar** di bagian bawah layar yang muncul secara otomatis dengan pesan **"Number 1 is shown!"**.
 - b. `LaunchedEffect` dieksekusi pertama kali saat `Composable` masuk ke dalam hirarki tampilan (*initial composition*). Karena variabel `number` terinisialisasi dengan nilai 1, `LaunchedEffect(number)` langsung berjalan dan memanggil `showSnackbar()`.
 - c. Pemicunya adalah pemanggilan fungsi `coroutine side-effect` `LaunchedEffect(number)`.

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!")
}
```
2. As
 - a. Nilai angka pada layar berubah menjadi **2**, lalu `Snackbar` sebelumnya berganti/diperbarui dengan pesan baru **"Number 2 is shown!"**.
 - b. Saat tombol diklik, callback `onButtonClick` mengubah nilai `state number` menjadi 2 (`number *= 2`). Perubahan nilai `number` ini memicu proses **recomposition**. Karena `number` dipasang sebagai *key* di dalam `LaunchedEffect(number)`, Jetpack Compose mendeteksi perubahan nilai pada *key* tersebut, lalu membatalkan (*cancel*) efek sebelumnya dan menjalankan ulang blok `LaunchedEffect` dengan nilai `number` yang baru (2).

Aktivitas Percobaan

- 1.

- a. Layar menampilkan angka **1** dan Snackbar di bagian bawah bertuliskan **"Number 1 is shown!"**.
 - b. Angka di layar secara otomatis berubah menjadi **angka acak antara 100–999** (misalnya 542), dan Snackbar secara otomatis diperbarui menampilkan pesan baru **"Number 542 is shown!"**.
 - c. Perubahan dipicu oleh berhasimnya panggilan fungsi `Repo.getData()` di dalam `LaunchedEffect(Unit)`. Ketika data berhasil didapatkan setelah *delay* 2 detik, variabel `number` diperbarui dengan nilai acak baru. Perubahan pada nilai `number` ini kemudian memicu **recomposition** serta memicu `LaunchedEffect(number)` untuk menampilkan Snackbar baru.
- ```
LaunchedEffect(Unit) {
 number = Repo.getData()
}
```

2.

- a. **LaunchedEffect(state)** menggunakan variabel *state* (seperti `number`) sebagai kuncinya (*key*). Efek di dalamnya akan dijalankan ulang setiap kali nilai *state* tersebut mengalami perubahan. Sedangkan **LaunchedEffect(Unit)** menggunakan konstanta `Unit` sebagai kuncinya. Efek ini tidak memiliki kunci yang dinamis, sehingga kodenya hanya dijalankan satu kali saja selama masa hidup (*lifecycle*) komponen Composable tersebut.
- b. **LaunchedEffect(Unit)** dijalankan **hanya satu kali** saat komponen Composable pertama kali dipasang ke dalam Tampilan (*initial composition*). Cocok untuk inisialisasi data awal seperti *fetch API*. Sedangkan **LaunchedEffect(state)** dijalankan pertama kali saat *initial composition*, dan akan **dibatalkan lalu dijalankan ulang (re-launched)** setiap kali nilai dari *state* yang menjadi kuncinya berubah.